Software Requirements Specification

Dine-In Restaurant Ordering and Inventory System

Design and Development Using Software Engineering Principles and Design Patterns

Team Members

Name	Registration Number
Shimul Das	2022331016
Md. Farhan Hasin Anik	2022331024
Syeda Maisha Anika	2022331026
Md. Pranta	2022331050
Abdullah Al Mamun	2022331080

Project Description

The Dine-In Restaurant Ordering and Inventory System is a web-based software application designed to automate and integrate the core internal operations of a restaurant, focusing exclusively on dine-in service. The system supports digital order taking, real-time kitchen coordination, inventory tracking, and billing.

Waiters create orders for specific tables and add menu items with required quantities. The system validates item availability before allowing items to be added. Once confirmed, orders are sent to the kitchen display where kitchen staff update preparation status. After food is served and payment is completed, the system automatically deducts ingredient quantities from inventory.

Managers can manually update stock levels, define low-stock thresholds, and view alerts through a dashboard when items fall below these limits. The system is developed following software engineering best practices and applies object-oriented design and appropriate software design patterns to ensure modularity, scalability, and maintainability.

Objectives

- To design a dine-in restaurant management system using software engineering methodologies.
- To implement a digital ordering workflow with real-time kitchen updates.
- To ensure accurate inventory tracking with availability validation and low-stock alerts.
- To apply object-oriented principles and multiple design patterns in system design.
- To develop a modular, maintainable, and scalable architecture.
- To gain practical experience in teamwork, documentation, and system modeling.

Software Requirements Specification

1. Introduction

1.1 Purpose

The purpose of this system is to build a dine-in restaurant management solution that digitizes ordering, kitchen coordination, billing, and inventory tracking, including availability validation and low-stock alerts.

1.2 Scope

The system will support:

- Table-based dine-in ordering
- Menu and item availability checking
- Kitchen order queue and status updates
- Billing and payment processing
- Automatic inventory deduction triggered by completed payment
- Low-stock alerts for management
- Manual inventory adjustments with audit logging
- Manager-administered staff accounts, with a role assigned at registration

Out of Scope: The following are explicitly excluded from this version:

- Online delivery or takeout
- Returns or refunds after payment
- Supplier or purchase order integration
- Cancellation or modification of orders after payment has been processed

2. Overall Description

2.1 User Roles

The system supports the following roles:

Module / Screen	Manager	Waiter	Kitchen	Cashier	Notes
Menu Management	Full	Read	–	–	Waiter can read to browse items
Table & Orders	Read	Full	Read	Read	Kitchen & Cashier view only
Kitchen Queue	–	–	Full	–	Kitchen updates status only
Billing & Payment	Read	–	–	Full	Cashier processes payment
Inventory	Full	–	–	–	Manager adjusts & sets thresholds
Reports & Alerts	Full	–	–	–	Dashboard visible to Manager only
User Management	Full	–	–	–	Pre-registered; no self-signup

2.2 Operating Environment

- Client: Angular single-page web application (SPA) with a role-based UI, accessed through a modern web browser
- Backend: Spring Boot REST API on Java 21

- Database: PostgreSQL, hosted on a managed serverless provider and reached over a TLS-required connection

2.3 Assumptions

- The system is designed for dine-in operations only
- Each menu item has a predefined recipe (ingredient mapping)
- Inventory is tracked at ingredient level
- Payment is recorded once per order
- All users are pre-registered in the system
- Client devices access the application over a local network or the internet through a web browser
- The system operates on a stable local or networked database connection

2.4 Order & Table Lifecycle

The order lifecycle is the backbone of the system: every requirement in Section 3 either advances an order along it or reads a value it produced. An order holds exactly one status at a time and shall move only forward along the transitions below. Any request for a transition that is not listed shall be rejected and shall leave the order unchanged.

Status	Meaning and permitted next state
PENDING	Open on a table and being built by the waiter. The only state in which lines may be added, changed, or removed (FR-07). Next: CONFIRMED.
CONFIRMED	Sent to the kitchen queue and visible on the kitchen display; the contents are locked (FR-09). Next: PREPARING.
PREPARING	The kitchen has started cooking (FR-12). Next: READY.
READY	The kitchen has finished; the order awaits delivery to the table and leaves the kitchen queue (FR-12). Next: SERVED.
SERVED	Delivered to the table (FR-10); the order is now billable (FR-13). Next: PAID.
PAID	Settled, the table released and stock deducted (FR-15, FR-19). Terminal: no further transition, edit, cancellation, or refund is permitted.

Table occupancy is a consequence of the order lifecycle rather than an independently editable value: creating an order occupies the table (FR-06) and payment releases it (FR-15). A table is AVAILABLE when it carries no open order and OCCUPIED while it does. NEEDS_SERVICE is a flag a waiter raises on an occupied table (FR-25); it does not affect the order status and does not free the table.

Ingredient stock is not reserved when an order is taken. The check at item-add time (FR-08) tests stock as it stands at that moment, and the single authoritative deduction occurs at payment (FR-19). Two orders may therefore each pass validation for the last portion of an ingredient; the first payment succeeds and the second is refused under FR-22. This is a deliberate decision: it keeps the deduction at one point in the system rather than splitting it between a reservation and a settlement.

3. Functional Requirements

3.1 Authentication & Authorization

ID	Requirement
FR-01	Users shall log in using a unique username and password.

ID	Requirement
FR-02	The system shall enforce role-based access control (RBAC) as defined in the access matrix in Section 2.1. Each role may only access screens and operations explicitly granted to it.

3.2 Menu Management (Manager)

ID	Requirement
FR-03	Manager shall create, update, and delete menu categories.
FR-04	Manager shall manage menu items (name, price, category, availability flag).
FR-05	If a waiter tries to add a menu item that isn't available, the system should stop them and show a clear message explaining that the item is currently unavailable.

3.3 Table & Order Management (Waiter)

ID	Requirement
FR-06	Waiter shall create a new order linked to a specific table. Creating an order shall set the table status to OCCUPIED.
FR-07	Waiter shall add menu items to an order, specifying quantity and optional notes per item, and shall change the quantity of, or remove, an existing line. Items may only be added, changed, or removed while the order status is PENDING (before confirmation). Adding a menu item that already appears on the order with identical notes shall increase the quantity of that existing line rather than create a second one. Each line shall record the menu item price in force at the moment the line was added, so that a subsequent change to the menu price does not alter the total of an order already taken.
FR-08	When a waiter attempts to add an item, the system shall validate: (a) the menu item is marked available, and (b) sufficient ingredient stock exists to fulfill the requested quantity. If validation fails, the system shall reject the item, display a descriptive error message, and leave the order unchanged. The stock check shall account for the quantity of each ingredient already committed by the other lines of the same order, not for the requested line in isolation. This check does not reserve stock (see Section 2.4).
FR-09	Waiter shall confirm an order and send it to the kitchen queue. Upon confirmation, the order status changes to CONFIRMED and no further item additions or removals are permitted.
FR-10	Waiter shall mark an order as SERVED after food has been delivered to the table.
FR-25	Waiter shall raise and clear a needs-service flag on an occupied table. The flag shall be visible to every role permitted to view the floor, and shall not affect the order status or the table occupancy (see Section 2.4).

3.4 Kitchen Operations (Kitchen Staff)

ID	Requirement
FR-11	Kitchen staff shall view the live order queue showing all orders in CONFIRMED or PREPARING status, ordered by confirmation time.
FR-12	Kitchen staff shall update an order's status to PREPARING and subsequently to READY. Orders persist in the queue until marked READY.

3.5 Billing & Payment (Cashier)

ID	Requirement
FR-13	Cashier shall view the full bill details for a served order, including all ordered items, quantities, and per-item prices.
FR-14	The system shall calculate and display the bill total broken down as: subtotal (sum of item prices × quantities), tax amount, service charge, and grand total.
FR-15	Cashier shall record the payment method and confirm payment. Upon successful payment: (a) the order status shall be updated to PAID, (b) the table status shall be reset to AVAILABLE, and (c) the system shall automatically trigger ingredient deduction as defined in FR-19. If that deduction cannot be completed because stock is insufficient (FR-22), the payment shall be refused in its entirety: no payment is recorded, the order remains SERVED, the table remains OCCUPIED, and no stock is deducted.
FR-16	The system shall generate a receipt upon payment containing: order ID, table number, date and time, list of items with quantity and unit price, subtotal, tax amount, service charge, and grand total. The receipt shall additionally record the payment method and the cashier who settled it, shall remain retrievable after payment so that it can be reprinted, and shall present the amounts and the rates that produced them as frozen at the moment of payment.

3.6 Inventory Management (Manager & System)

ID	Requirement
FR-17	The system shall maintain an ingredient inventory record for each ingredient, including current stock quantity and unit of measurement.
FR-18	Manager shall set a low-stock threshold per ingredient. When current stock falls at or below the threshold, the system shall generate a low-stock alert.
FR-19	Upon successful payment (triggered by FR-15), the system shall automatically deduct ingredient quantities from inventory based on the recipe mappings of all items in the order. This deduction shall be performed as a single atomic database transaction.
FR-20	The manager dashboard shall display all ingredients currently at or below their low-stock threshold, including ingredient name, current stock, and threshold value.
FR-21	Manager shall manually adjust inventory stock levels for any ingredient. Each adjustment shall be recorded in an audit log capturing: manager ID, ingredient ID, quantity changed (positive or negative), reason, and timestamp.
FR-22	The system shall not allow any inventory deduction — automated or manual — to reduce an ingredient's stock below zero. If a deduction would result in negative stock, the system shall halt the operation and notify the responsible party.

3.7 Reporting (Manager)

ID	Requirement
FR-23	The manager shall view a sales summary for a specified date range, including total revenue, number of orders completed, and breakdown by menu category. The date range shall be interpreted as inclusive calendar days in the restaurant's configured timezone. All figures shall be derived from recorded payments rather than recomputed from current menu prices, so that a later price or rate change cannot alter a past report.

ID	Requirement
FR-24	The manager shall view the top-selling menu items ranked by quantity sold within a specified date range. The date range is interpreted as in FR-23.

3.8 User Management (Manager)

ID	Requirement
FR-26	Manager shall register a staff account with a username, a display name, and a role, and shall update the display name and role of an existing account. There is no self-registration; an account can be created only by a Manager.
FR-27	Manager shall reset the password of any staff account. Stored passwords shall never be displayed or recoverable; replacement is the only supported operation.
FR-28	Manager shall disable and re-enable a staff account. Accounts shall not be deleted, so that the orders, payments, and audit records attributed to them remain intact. Disabling an account, or changing its role, shall take effect on that account's next request rather than when its existing session expires.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	Each role-based UI must be operable by staff with minimal training. Critical workflows (adding an item, updating kitchen status, processing payment) shall be completable in 3 steps or fewer.
NFR-02	Performance	The kitchen order queue shall refresh within 3 seconds of an order being confirmed. This applies on a stable local area network. Near-real-time updates via short polling or a push channel (WebSocket / Server-Sent Events) are acceptable implementation strategies for the browser client.
NFR-03	Security	User passwords shall be stored as salted hashes. Authentication shall use JWT bearer tokens. The backend REST API shall enforce RBAC on every endpoint, rejecting unauthorized requests with an appropriate HTTP error response regardless of client-side controls. Authorization shall be re-evaluated from the stored user record on every request, so that disabling an account or changing its role takes effect on the next request rather than when the existing token expires.
NFR-04	Reliability	Inventory deductions and payment recording shall be executed within a single database transaction. If any step fails, the entire transaction shall be rolled back to prevent partial updates. Stock levels shall never fall below zero (see FR-22).
NFR-05	Maintainability	The system shall follow a layered architecture (presentation, service, repository) with clearly separated concerns. Software design patterns shall be applied where appropriate to support modularity and extensibility. Conformance to the layering shall be verified automatically by the build rather than by review alone.
NFR-06	Auditability	All manual inventory adjustments shall be persisted in an immutable audit log (see FR-21). Audit records shall not be editable or deletable through any user interface or API route, and immutability shall additionally be enforced by the

ID	Category	Requirement
		database itself, so that no direct SQL statement can amend or remove a recorded adjustment.
NFR-07	Concurrency	The system shall handle simultaneous requests from multiple users without data corruption. Concurrent inventory deductions for the same ingredient shall be serialized using database-level locking (e.g., PostgreSQL row-level locks via SELECT ... FOR UPDATE) to ensure consistency.

5. External Interfaces

5.1 User Interface

Angular-based single-page web application (SPA), accessed through a web browser, with role-specific dashboards:

- Manager Dashboard — inventory alerts, reports, menu and inventory management
- Waiter Order Screen — table selection, order creation, item addition, order confirmation
- Kitchen Queue Screen — live order queue with status update controls
- Cashier Billing Screen — bill details, payment recording, receipt generation

5.2 APIs

RESTful APIs built using Spring Boot. All communication between the Angular client and the server uses JSON over HTTP. Authentication is performed with JWT bearer tokens, and Cross-Origin Resource Sharing (CORS) is configured to allow the browser-based client. Every API endpoint enforces role-based access control server-side.

5.3 Hardware Interfaces

- Any device with a modern web browser (desktop or laptop)
- Optional: Receipt printer for physical invoice output

5.4 Database

- PostgreSQL relational database
- Structured schema with defined relationships across Orders, Items, Inventory, Recipes, and Audit tables
- Database migrations managed with Flyway, versioned in the repository and applied automatically at startup; the ORM validates the schema against the migrations and never modifies it
- A CHECK constraint enforces non-negative ingredient stock at the database level (see FR-22), and row-level locking (SELECT ... FOR UPDATE) serializes concurrent inventory deductions (see NFR-07)

6. Constraints & Assumptions

- System is limited to dine-in functionality only
- Waiters may add or remove items from an order only while it is in PENDING status (before confirmation). No modification is permitted after the order is sent to the kitchen
- No refund, cancellation, or modification is permitted after payment has been processed
- Inventory deductions occur automatically upon successful payment as a single atomic transaction
- Requires a stable local or networked database connection for all operations

- Uses a shared development database with migration version control
- All users must be pre-registered by a Manager (FR-26); self-registration is not supported

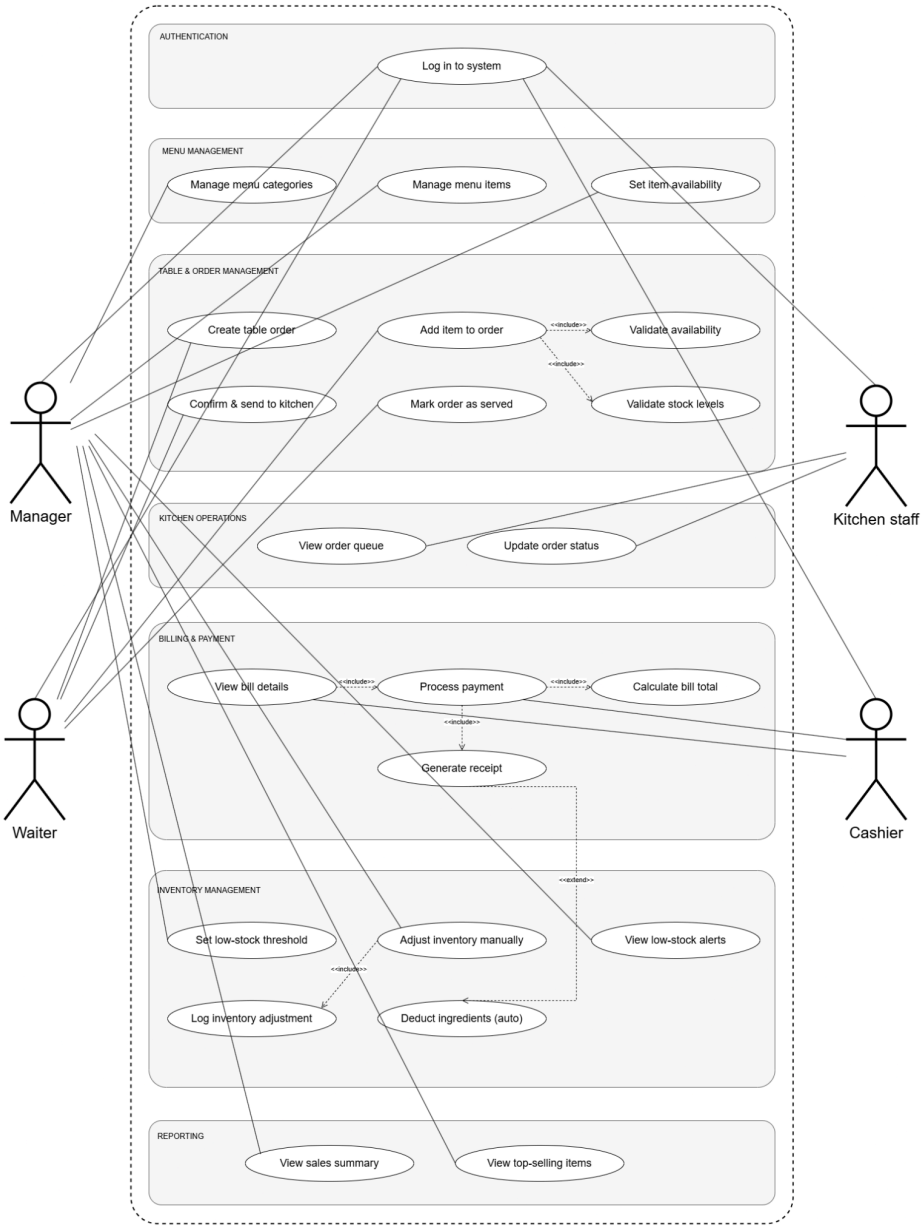
7. Appendices

7.1 Glossary

Term	Definition
Order	A collection of one or more menu items requested by a specific table, associated with a single waiter.
OrderItem	An individual menu item line within an Order, including quantity, unit price, and optional notes.
OrderStatus	The lifecycle state of an Order. Valid states: PENDING → CONFIRMED → PREPARING → READY → SERVED → PAID.
Bill	A financial document generated from a SERVED Order, containing the itemized breakdown, subtotal, tax, service charge, and grand total.
Payment	A record of a completed financial transaction against a Bill, capturing the payment method and timestamp. Triggers inventory deduction.
PaymentMethod	The means by which a bill was settled. Supported values: CASH, CARD, MOBILE.
Ingredient	A raw material tracked in inventory, used in one or more recipes. Tracked by stock quantity, unit, and low-stock threshold.
RecipeItem	A mapping between a MenuItem and an Ingredient, specifying the quantity of that ingredient required per unit of the menu item.
InventoryAdjustment	A manual stock-level change made by a Manager, recorded in the audit log with reason and timestamp.
TableStatus	The occupancy state of a RestaurantTable. Valid states: AVAILABLE, OCCUPIED, NEEDS_SERVICE.
LowStockAlert	A system-generated notification triggered when an ingredient's current stock falls at or below its configured threshold.
RBAC	Role-Based Access Control. A security model where system permissions are granted based on the user's assigned role.
Audit Log	An immutable record of all manual inventory adjustments, capturing who made the change, what changed, by how much, why, and when.

8. Use Case Diagram

The use case diagram below is unchanged by the platform migration; it models actors and system behaviour independently of the client technology.



9. Class Diagram

The class diagram below is likewise unchanged. It defines the object-oriented domain model — including the repository interfaces, entity classes, and enumerations — that the Spring Boot backend implements regardless of the frontend framework.